One Step Ahead

A friction-free micro-breakdown engine

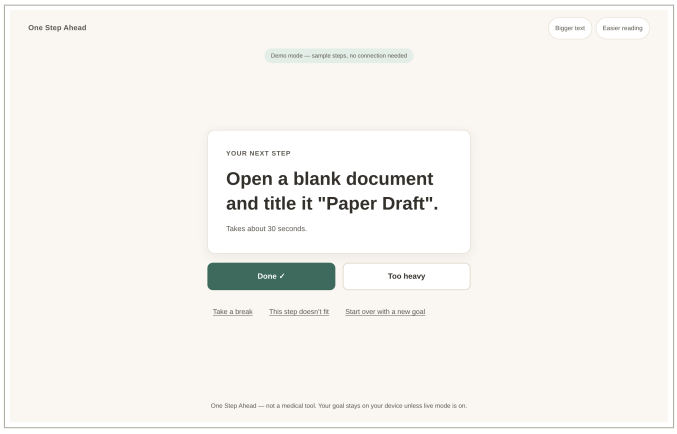
Demo and Walkthrough Report

Maissara Guersas

TaskForge

October 24, 2026

Abstract. One Step Ahead is a web tool for people who freeze when a task feels too big: instead of generating a plan, it surfaces exactly one tiny, concrete action — under two minutes, near-zero decisions — and shows nothing else. This report demonstrates the working product through real screenshots captured from the running application, following one composite user, Maya, from an avoided research paper to a finished coding goal. Two mechanisms distinguish the tool: a strict one-step invariant (no future step, no total, and no list is ever present in the UI or the DOM) and an infinitely repeatable shrink ladder that makes any step smaller, all the way down to a single breath. The report documents the walkthrough, the accessibility decisions (WCAG 2.2 AA), the LLM behavior contract, the validation and fallback chain, and the tests that were actually executed: 73 unit checks, 28 real-browser end-to-end checks, and 19 verified screenshots. Maya is a fictional composite persona, no user studies have been run, and every screenshot shows at most one step at a time.



The product in one screen: exactly one step, and nothing else (demo mode, 1440×900; the walkthrough begins in Section 4).

Contents

1 Introduction **3**

2 The Problem: Why a 10-Step Checklist Makes It Worse **3**

3 The Solution: One Step, Ever **3**

4 Walkthrough **4**
- 4.1 Starting: One Question, One Button . . . 4
- 4.2 One Step at a Time . . . 4
- 4.3 Smaller, and Smaller, and Smaller . . . 5
- 4.4 Done — and a Second Domain . . . 5
- 4.5 Taking a Break, and Completion . . . 6

5 Before and After **6**

6 Accessibility **7**

7 Technical Design **8**
- 7.1 Architecture: Deliberately Boring . . . 8
- 7.2 The One-Step Contract . . . 8
- 7.3 Prompt Design: Shrinking on Request . . . 9
- 7.4 Validation and the Fallback Chain . . . 9
- 7.5 Two Modes, One Promise . . . 10

8 Testing **10**

9 Limitations **11**

10 What We Learned and What's Next **12**

11 AI Assistance and Pre-existing Work Disclosure **12**

12 Team **12**

A How to Run the Project **12**

1 Introduction

Maya is 24, a graduate student with ADHD, and she has a research paper due on Friday. Today she has opened her laptop four times. Each time she looks at the words "Write research paper," feels a wall of dread come down, and closes the lid again. Nothing is wrong with her intelligence, her skills, or her intentions; what is broken is the handoff between *wanting* to do the thing and *starting* the thing.

A transparency note: Maya is a composite persona, built from widely documented experiences of ADHD and executive dysfunction — she is fictional. This report contains no real quotes, no user data, and no testimonials, because no user studies have been run; every statement about what she would feel is design intent, labeled as such in Section 5.

The change we aim for is narrow and concrete: from *frozen* to *started* in under a minute. One Step Ahead asks one question ("What's the big thing you're avoiding?"), then shows exactly one micro-step — a single, physical or digital action that takes under two minutes and requires no deciding and no planning — and when a step still feels too heavy, one tap makes it smaller, down to "Breathe." The application is plain HTML, CSS, and vanilla JavaScript with a 55-line proxy for live mode and a fully offline demo engine; it runs by opening `index.html`. The rest of this report walks through the real product, screen by screen, then documents the design and the tests behind it.

2 The Problem: Why a 10-Step Checklist Makes It Worse

The barrier is executive-dysfunction paralysis — the freeze that happens when a person can see the size of a task. It affects people with ADHD and depression, chronic procrastinators, and anyone who has reorganized a desk to avoid writing a first sentence. The paralysis is not a knowledge problem: Maya knows perfectly well what writing a paper involves. It is an activation problem, and the moment the whole mountain is visible, starting becomes physically hard.

Almost every tool built for this moment is optimized for planning, not starting. Task apps ask Maya to break the paper into subtasks — which is itself a task she cannot start. AI planners are worse: they return a tidy ten-step checklist in seconds, which for a brain that freezes at the sight of the mountain is the inside of her head rendered on a screen: the mountain, itemized. Time-to-first-action gets longer, not shorter, because now there is also a plan to feel guilty about ignoring. The design conclusion: a tool for activation paralysis must not render the task's full structure — it must compress the visible future to one tiny action, and be able to shrink that action without limit, because "just open the document" is sometimes still too heavy on a bad day.

3 The Solution: One Step, Ever

One Step Ahead is deliberately almost empty. The start screen is one question and one button. The step screen shows one micro-step in large, calm, plain language, with two primary buttons: **Done ✓** (get the next single step) and **Too heavy** (get a strictly smaller one). Under the surface sit three quiet support actions — "Take a break," "This step doesn't fit," and "Start over with a new goal" — and nothing else: no project tree, no calendar, no streaks.

Design principle: the one-step invariant

The user never sees a future step, a step total, an "X of Y", or a full list — not in the UI, not in the DOM, not in the console. There is no hidden container of upcoming steps and no

"show all steps" feature. A rejected step is replaced, never stacked. This is enforced in code and covered by automated static and browser checks (Section 8).

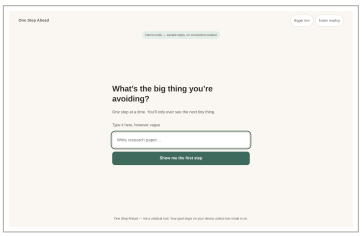
Everything else follows from that invariant. A step is valid only if it is one sentence of at most 20 words that starts with a verb and names one concrete, observable action needing no decision — "Open a blank document and title it 'Paper Draft'." is a step; "research the topic" is not. Steps carry their own duration estimate, capped at two minutes, rendered as a static line ("Takes about 30 seconds."), never a ticking timer. The shrink ladder is the second half of the design: "Too heavy" can be pressed any number of times, and the result must be strictly easier every time, with "Breathe." as the bedrock (Figure 3). Feedback stays gentle: after each Done, one short warm line ("Rough counts.") and a tally that only ever counts up. When the goal is judged finished, a calm screen says so.

4 Walkthrough

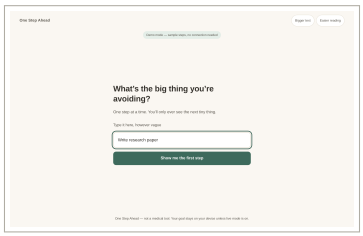
All screenshots are real captures of the running application, taken with headless Chromium (Playwright) at 1440×900 (desktop) or 390×844 (mobile), device scale factor 2, no browser chrome, cropped consistently. The walkthrough runs in *demo mode* — the built-in offline engine — because it is deterministic and reproducible with no key and no network; the badge in the UI always says so. The live path and the failure paths appear in Section 7.

4.1 Starting: One Question, One Button

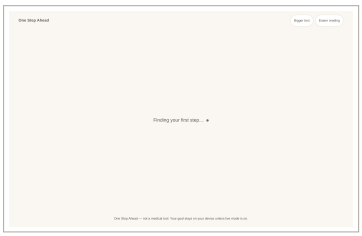
The product surface is one question, one labeled input, one button (Figure 1). The input accepts anything, however vague — an empty submission still returns a step. The header carries the only two toggles in the app, **Bigger text** and **Easier reading**, which persist across visits. Maya types her goal and presses Enter; while the next step is being found, the app shows one calm line and nothing that moves, and a wait beyond 15 seconds switches to a friendly retry screen (Figure 9).



(a) Empty.



(b) The goal, typed.



(c) Finding the first step…

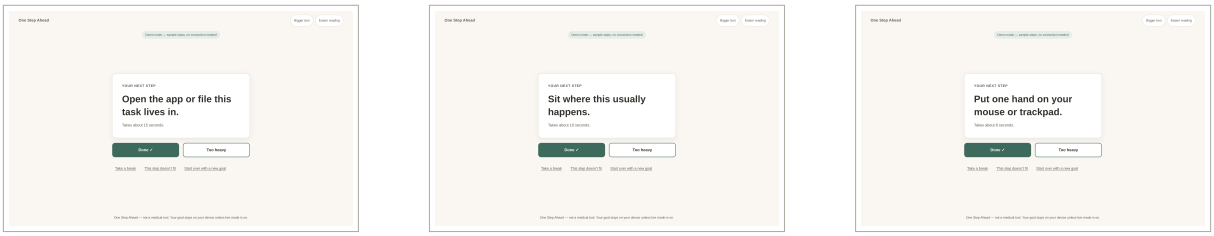
Figure 1: Getting started. Notice what is absent in (a) — no onboarding, no examples gallery, no sign-in; the only choice is to begin. In (b) the input keeps keyboard focus (the soft ring is the focus indicator); Enter submits. (c) The loading state quotes one line and never spins.

4.2 One Step at a Time

The first step appears: open a blank document and title it — the hero image on the title page. Thirty seconds, near-zero decisions, nothing else on the screen. Because the walkthrough runs offline, the badge explains the mode honestly; in live mode the badge is absent. Pressing Enter anywhere on this screen counts as Done, so the whole flow can be driven from the keyboard.

4.3 Smaller, and Smaller, and Smaller

Suppose even this is too heavy today. Maya presses **Too heavy** three times, and each press returns a strictly easier step (Figure 2): first the step keeps the goal's neighborhood but drops all creating — just open, find, or look; then it becomes body-only setup, where sitting down is the whole step; and then a purely physical action, from a level where every rung is something a body can do without a single decision.



(a) Once. **(b)** Twice. **(c)** Three times.

Figure 2: "Too heavy" three times: (a) "Open the app or file this task lives in." (the encouragement line, "Smaller is allowed.", is deliberately permissive), (b) "Sit where this usually happens.", (c) "Put one hand on your mouse or trackpad."

Figure 3 shows the full deterministic ladder the demo engine uses; the live mode receives the same semantics through the prompt (the rejected step plus a shrink level), and the validator refuses any replacement that repeats the rejected step. The ladder has a bedrock, so there is no floor the user can fall through.

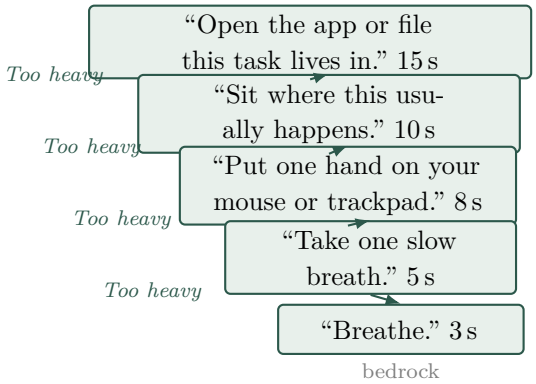
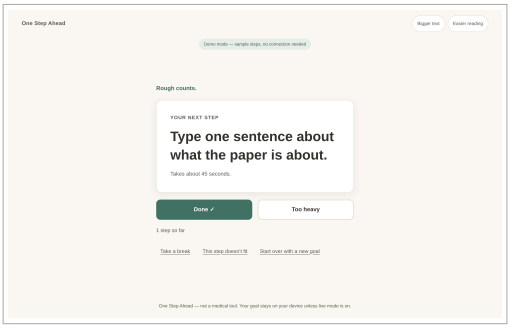


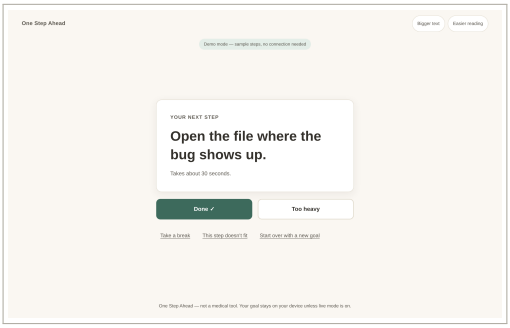
Figure 3: The shrink ladder (demo engine, abbreviated). Each press of "Too heavy" moves strictly down; the boxes shrink with the steps. Nothing above the current rung is shown to the user, and no future step appears anywhere.

4.4 Done — and a Second Domain

From the rung she landed on, Maya opens the file and presses **Done ✓**. The response is one warm line ("Rough counts.") and the next single step, with a tally underneath that counts only what is already finished and can only ever grow — there is no denominator, because a denominator is a mountain (Figure 4a). The engine is also not writing-specific: starting over with "Fix bug in auth service" produces the same shape, a tiny concrete door-opener for a coding session (Figure 4b). The demo engine ships hand-written breakdowns for three sample domains (a paper, a bug, a dreaded email) plus a generic ladder; in live mode the model generalizes under the same one-step contract. "Start over" is a two-press action, so a stray click cannot wipe a session.

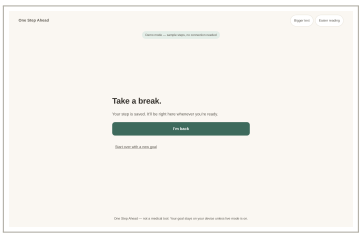


(a) After Done: acknowledgment, next step, tally.

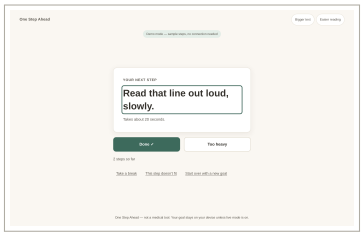


(b) A different domain, the same shape.

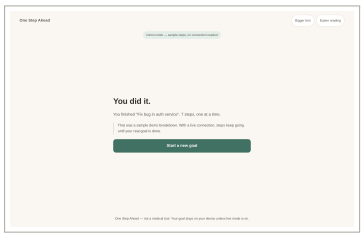
Figure 4: Momentum, gently. (a) An acknowledgment line ("Rough counts."), the next single step, and a tally that only counts up — no "X of Y", no streak, no progress bar. (b) The first step for a coding goal: open the file where the bug shows up.



(a) The break screen.



(b) After reload: same step, tally intact.



(c) "You did it."

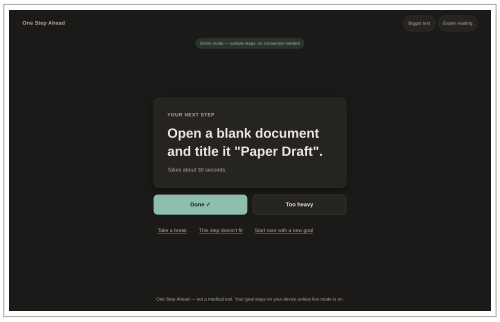
Figure 5: Exit and ending. (a) One message, one button. (b) Take a break, close the laptop, reload — the session survives; the step after resuming is the same one that was waiting before the break (tally: "2 steps so far"). (c) The completion screen: calm, specific, terminal — no confetti, no score, just permission to stop.

4.5 Taking a Break, and Completion

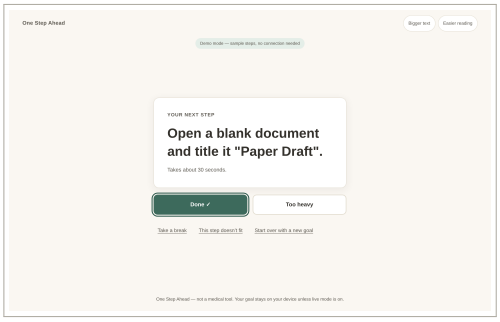
Paralysis tools need an honorable exit. "Take a break" (Figure 5a) saves the whole session — goal, current step, tally — in `localStorage` (wrapped in try/catch), so a full page reload lands on the same screen and "I'm back" returns to exactly the same step (Figure 5b): the work of remembering is done by the app, not the person. When the last step of the breakdown is done, the app does not celebrate; it names the goal, counts the steps quietly, and offers exactly one button — start a new one (Figure 5c). In demo mode, a note explains that a live session would keep generating steps until the model judged the real goal finished.

5 Before and After

Table 1 contrasts the familiar planning experience with One Step Ahead. One caveat governs the whole table: no user studies have been run, so every entry — especially the "feels" row — is design intent, what the product is built to produce, not a measured result.



(a) Dark mode, following the system preference.



(b) Tab reaches Done, with a thick offset focus ring.

Figure 6: Accessibility in practice. (a) The step screen in dark mode; contrast pairs in this theme pass 4.5:1, verified numerically by the test suite. (b) Keyboard navigation: the **Done** ✓ button carries the focus ring, and the skip link has appeared at the top left because it is focused.

Table 1: Typical AI planner vs. One Step Ahead. All entries are design intent, not measured outcomes; no user studies have been conducted.

	Typical AI planner	**One Step Ahead**
Time to first action	5–20 minutes of generating and reading a plan before any work begins	Designed for under a minute: type one line, get one step
What she sees	"10 steps to write your research paper: 1. Choose topic… 2. Gather sources…"	"Open a blank document and title it 'Paper Draft'."
What she feels (intent)	The whole mountain, itemized; dread with a progress bar	"That's… it? I can do that."
When it is still too big	Re-plan, or push through the guilt	One tap: "Too heavy" → "Sit where this usually happens."
After five opens today	Zero progress, plus a plan to feel bad about	A file exists, titled "Paper Draft"

6 Accessibility

Accessibility was a floor, not a stretch goal, and the target was WCAG 2.2 AA. Every text/background pair in both the light and the dark theme passes 4.5:1 contrast — verified numerically by the test suite, which parses the CSS and computes the ratios (Section 8). The palette is soft neutrals with one muted pine accent; there is no red anywhere, no animation that cannot be turned off, and no countdown timers. The app follows system preferences for theme and motion: `prefers-color-scheme` switches a calm dark palette (Figure 6a), and `prefers-reduced-motion` disables the single 0.25 s screen fade.

Keyboard operation is complete — Enter submits the goal, Enter on the step screen counts as Done — and every interactive element carries a visible 3 px focus ring with offset (Figure 6b). Screen readers get semantic HTML with one `h1` per screen, a labeled input, a skip link (the first Tab stop, visible only when focused), and polite live-region announcements when the step changes.

Reading was treated as part of accessibility. The step text is 40 px on desktop and still 26 px at a 320 px viewport; the layout is responsive from 320 px up (Figure 7); touch targets are at least 48 px, with primary buttons at 56 px. Two quiet toggles sit in the header: **Bigger text** and **Easier reading**, a dyslexia-friendlier font stack with increased letter and word spacing, implemented in CSS so the app stays offline and dependency-free.

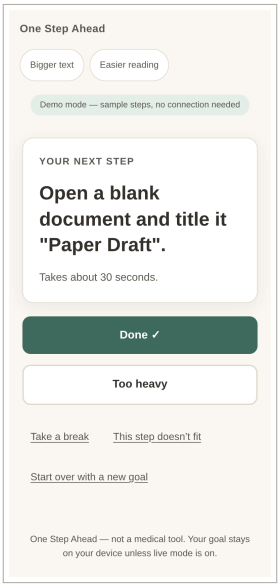


Figure 7: Mobile view of the step screen (390×844): the step text stays large, the buttons stack, and the one-step layout survives the narrow viewport; the layout is responsive from 320 px up.

7 Technical Design

7.1 Architecture: Deliberately Boring

The stack optimizes for auditability and zero install: plain HTML, CSS, and vanilla JavaScript in the browser; a 55-line Node proxy (`server.js`) for live mode; and a deterministic offline demo engine (`demo-engine.js`) as both the no-key experience and the safe-step fallback (Figure 8). There is no framework, no build step, and no npm dependency — the proxy uses only `http`, `fs`, and Node's built-in `fetch`, so Node 18+ is the only requirement. The API key is read only from the environment — never committed, never sent to the browser — and the temperature is fixed server-side at 0.4 for consistency. The default model is `claude-sonnet-4-5`; port and model are configurable.

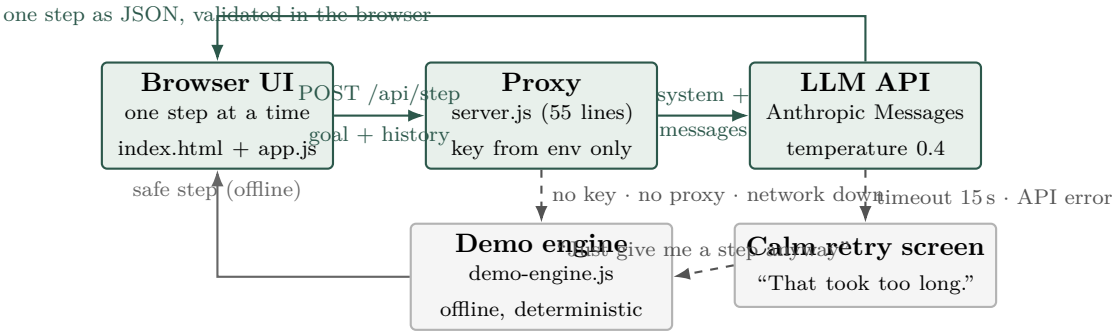


Figure 8: System architecture with the demo-mode fallback path. Solid arrows carry the live path; dashed arrows are degradations that never dead-end. One further fallback is not drawn: a reply that still fails validation after two corrective retries is replaced by a built-in safe step from the demo engine.

7.2 The One-Step Contract

The model never returns free text. It must reply with exactly one JSON object (Listing 1) — no preamble, no markdown, no lists — or the reply is treated as invalid and retried. The persona, the size rule, the shrink semantics, and a forbidden list ("plan", "research", "figure out", "work on"; future steps; multi-part steps; pressure language; emoji) live in a system prompt

that is an editable constant in `prompts.js`, together with four few-shot examples covering a writing task, a coding task, a household task, and an emotional-admin task, plus a completion example. Field rules are re-enforced in the browser: `step` is one sentence of at most 20 words that starts with a verb and names one concrete action; `estimated_seconds` lies between 1 and 120; `is_goal_complete` is true only when the completed history shows the goal is genuinely finished; `encouragement` is optional and at most 12 words.

Listing 1: The model's entire reply shape: one micro-step as strict JSON.

```
{"step":"Open a blank document and title it 'Paper Draft'.",
 "estimated_seconds":30,
 "is_goal_complete":false,
 "encouragement":"That's started."}
```

7.3 Prompt Design: Shrinking on Request

Each user action maps to one request template, and the most interesting one is the shrink request (Listing 2), because "strictly smaller" must be a property of the *replacement*, not a hope. The template sends the rejected step and the current shrink level; the system prompt defines the levels; the validator refuses a replacement that repeats the rejected step. Conversation context is capped at the last eight completed steps to keep tokens and latency low.

Listing 2: The SMALLER_STEP template (excerpt, verbatim from `prompts.js`).

```
GOAL: {goal}

THE PERSON SAID THIS STEP WAS TOO HEAVY:
"{rejected_step}"

TIMES IT HAS ALREADY BEEN SHRUNK: {shrink_level}

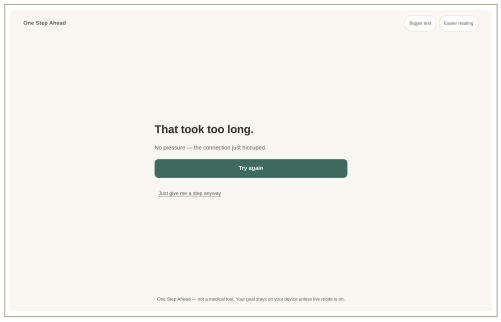
Reply with ONE step that is strictly easier and smaller than the
rejected one, as JSON only.
Shrink guidance - level 1: just open, find, or look (no typing, no
creating). Level 2: body-only setup (sit down, bring it within reach).
Level 3 or higher: pure physical or sensory (put a hand on the
trackpad, take one breath).
Never repeat the rejected step.
```

7.4 Validation and the Fallback Chain

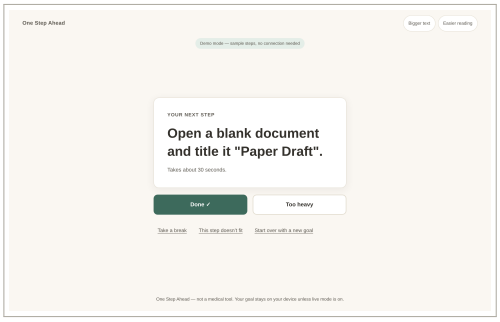
Every reply is schema-validated in the browser before it is shown (Listing 3). An invalid reply triggers up to two corrective retries — the model's own output is sent back with the reason it failed — and if the reply still fails, a built-in safe step is served from the demo engine. Around the model sit the degradations: no key, no proxy, or a dead network each switch the app to demo mode silently; a timeout (15 seconds) or an API error produces the calm retry screen (Figure 9a); and the retry screen itself offers "Just give me a step anyway," which engages demo mode (Figure 9b). The app never dead-ends, and the badge always tells the truth about which engine is talking.

Listing 3: Validator excerpt (`prompts.js`, abridged): the rules every model reply must pass.

```
if (words.length > 20) errors.push('step is longer than 20 words');
if (/and\s+then/i.test(s)) errors.push('step must not contain "and then"');
if (/(^|\n)\s*\d+[.)]\s/.test(s))
```



(a) The calm retry screen.



(b) Fallback engaged: demo mode, badged.

Figure 9: An API failure (here: an endpoint returning HTTP 500, driven through the app's real error path) first shows a screen with no blame and no jargon, then — on request — falls back to the offline engine. Both screens were produced by the app's own code; see Section 8 for the harness disclosure.

```
  errors.push('step must not contain a numbered list');
if (/\bstep\s*\d+/i.test(s) || /\bnext step\b/i.test(s))
  errors.push('step must not mention future steps');
if (!opts.relaxVerbs && words.length) {
  var first = words[0].toLowerCase().replace(/[^a-z]/g, '');
  if (first && !VERBS[first])
    errors.push('step must start with a verb');
}
```

7.5 Two Modes, One Promise

Live mode and demo mode render through the same screens, so the one-step promise holds either way; the only visible difference is the badge, which appears whenever the offline engine is serving steps and is absent on the live route. State is kept in memory and mirrored to `localStorage` inside try/catch, so a refresh — or "Take a break" — resumes the exact session, as demonstrated in Figure 5.

8 Testing

This section reports only tests that were actually executed for this report, with results copied from the runs (Table 2); every screenshot in this document was re-verified before use.

Table 2: Test suites executed for this report, from the project root (Node 18+).

Command	Covers	Result
`node test.js`	validator, reply parsing, retry/fallback paths, demo engine across many goals (empty, emoji, non-English, 2000-char, prompt-injection), persistence, WCAG contrast for both themes, static safety (no DOM lists, no console output, no committed keys, proxy under 60 lines), full simulated session	73 / 73
`bash e2e.sh`	real-browser end-to-end regression on the running server: all flows, "Too heavy" five times (strictly smaller), ack and tally, alternate, reload and break/resume, two-press restart, an email goal to completion, hostile inputs each returning exactly one step, console cleanliness	28 / 28
curl smoke	`GET /` → 200; missing file → 404; traversal → 403; `POST /api/step` without a key → 502 with a JSON error	as expected
screenshot verification	all 19 captures: correct dimensions, non-blank, content checked image by image (this report's Loop A)	19 / 19

Quoted from the actual runs:

```
$ node test.js
PASSED: 73   FAILED: 0

$ bash e2e.sh
  ok   zero page errors
E2E PASSED: 28  FAILED: 0

$ curl -s -w "%{http_code}" -X POST localhost:3000/api/step ...
{"error":"No API key configured on the server (set ANTHROPIC_API_KEY)."} 502
```

Two disclosures keep this section honest. First, the live path was not exercised against a real Anthropic key, because none exists in this build environment; the retry and fallback logic around the model is covered by unit tests with a fake API and by the browser error-path captures. Second, the transient states in Figures 1 and 9 (loading, and the API-failure path) were photographed through a small local harness (`scripts/mock-server.js`, included in the submission) that delays or fails the `/api/step` endpoint; every screen shown is the app's real DOM on its real code paths.

9 Limitations

Limitation: not a medical tool

One Step Ahead is a starting aid. It is not a medical device, not therapy, and no substitute for professional support or treatment.

Beyond that headline, the honest list: live-mode quality depends on the LLM's adherence to the one-step contract; validation and fallbacks mitigate but cannot eliminate a bad-reply cycle. The product is English-first: steps are generated in the goal's language when live, but the verb-start check is English-only (relaxed for non-Latin scripts), and the demo engine is English-only. No user studies have been run — every claim about time-to-first-action and every "feels" entry in Table 1 is a design target, not a measurement. Demo mode offers three curated sample breakdowns plus a generic ladder; its "completion" is the end of a sample, labeled as such in the UI. And for privacy: goals and history stay on the device, but in live mode the goal text and the

last few completed steps are sent to the Anthropic API through the operator's own proxy; there are no analytics, no accounts, and no tracking.

10 What We Learned and What's Next

Three lessons stand out. Smallness is a feature that has to be defended: every review pass found something tempting to add — progress bars, streaks, step counts — each cut against a single test: does it reduce the person's load? Prompts are UX: the product's personality lives as much in a JSON schema and a forbidden-verb list as in the interface. And fallbacks are the product: the no-key and timeout paths received as much design and testing effort as the happy path. Skeptical testing paid for itself: the memorable bugs (a cross-origin abort masquerading as a network error; a Python static server answering `POST` with 501, now mapped to "no proxy here") were only visible by running the app.

What's next, in order of importance: run real user studies with ADHD communities (with consent and oversight) to test whether one-step-first actually reduces time-to-start; non-English verb validation and localized demo ladders; an on-device or small-model mode for fully private step generation; a new-tab browser extension, so the start screen lives where the paralysis already lives; and gentle, local-only history with no streak mechanics.

11 AI Assistance and Pre-existing Work Disclosure

This project was built with AI coding assistance: an AI coding agent was used for scaffolding, code, tests, and documentation; all code was reviewed and executed before inclusion, and the test results quoted in Section 8 come from actual runs in this environment. This report was drafted with AI assistance, from the project's real files and captures.

The product calls an LLM API (Anthropic's Claude, via the operator's own proxy and key) in live mode; demo mode is entirely deterministic, hand-written content. No pre-existing code beyond the toolchain was used (zero npm dependencies, system font stacks). The report itself was produced with Playwright (Chromium) captures, vision-model image verification, and Tectonic (LaTeX) typesetting.

12 Team

One Step Ahead is a solo submission to TaskForge by Maissara Guersas: the product brief — the one-step rule, the Maya persona, the accessibility bar, the zero-dependency constraint — is the author's, and the implementation, tests, Devpost package, and this report were produced from it with the AI assistance disclosed in Section 11. The submission is the self-contained source tree; the Appendix gives the exact commands to run the app, the tests, and this report locally.

A How to Run the Project

No build step, no dependencies. From the project root (`one-step-ahead/`), demo mode needs no key: double-click `index.html` or serve the folder statically (`python3 -m http.server 8000`). For live mode (Node 18+), copy `.env.example` to `.env`, set the key, run `npm start`, and open `localhost:3000`. The tests quoted in Section 8:

Listing 4: Tests and report rebuild.

```
npm test                            # 73 unit checks
node server.js & bash e2e.sh        # 28 real-browser checks (needs :3000)
tectonic report.tex                 # run twice (resolves TOC/refs)
```